White Box Report

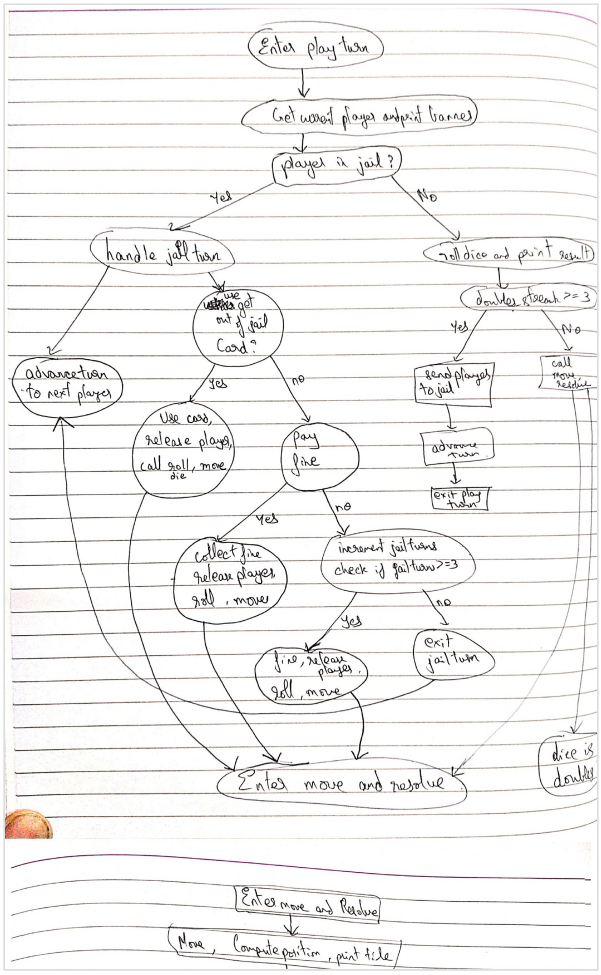
1.1 Control Flow Graph

The control flow graph was derived from the principal execution path of the MoneyPoly program. The graph started at `main()`, where player names were collected and the `Game` object was created, and then followed the main gameplay flow through `Game.run()`, `play_turn()`, and `_move_and_resolve()`. The branches inside `_handle_jail_turn()`, `_apply_card()`, and `_handle_property_tile()` were expanded because those functions contained the most important decisions, state changes, and branch paths in the game.

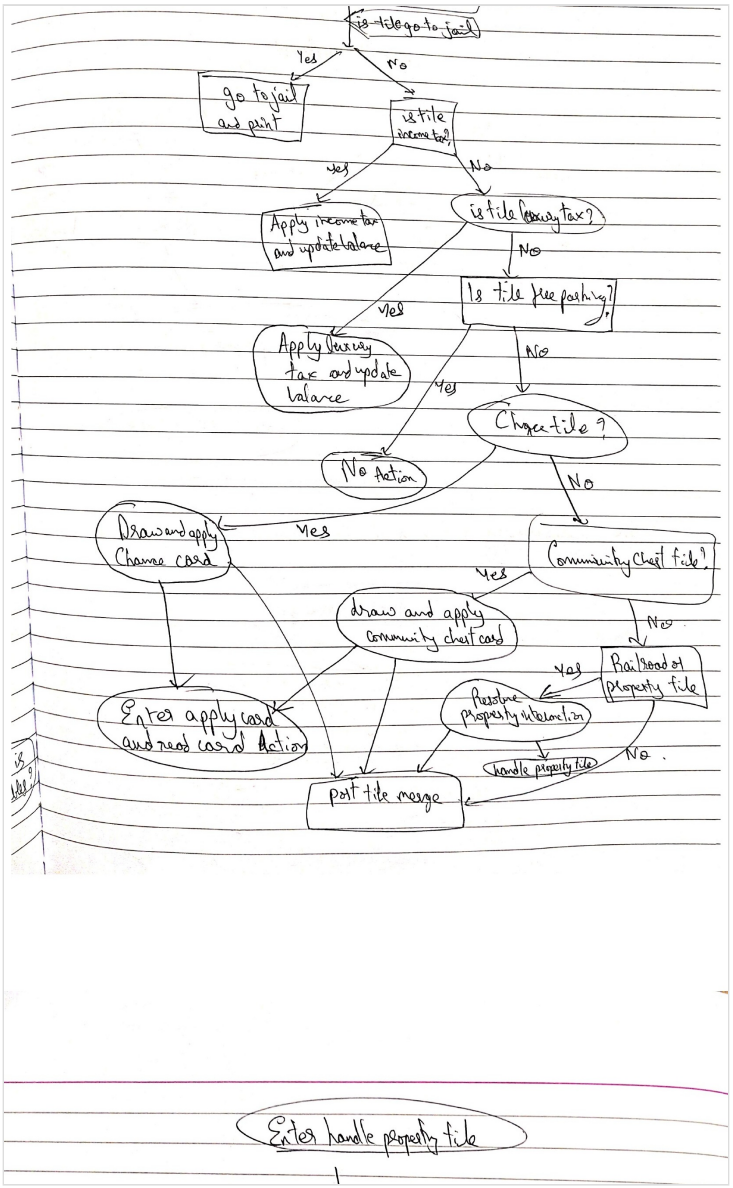
To keep the handwritten CFG readable, sequential statements were grouped into related blocks and repeated post-branch flows were merged into common nodes while preserving the real control structure. The submitted diagram was created from the actual code and labeled with clear node and edge names.

Because the handwritten CFG was taller than a single report page, it was embedded below as four ordered slices so that the complete diagram remained readable in the PDF.

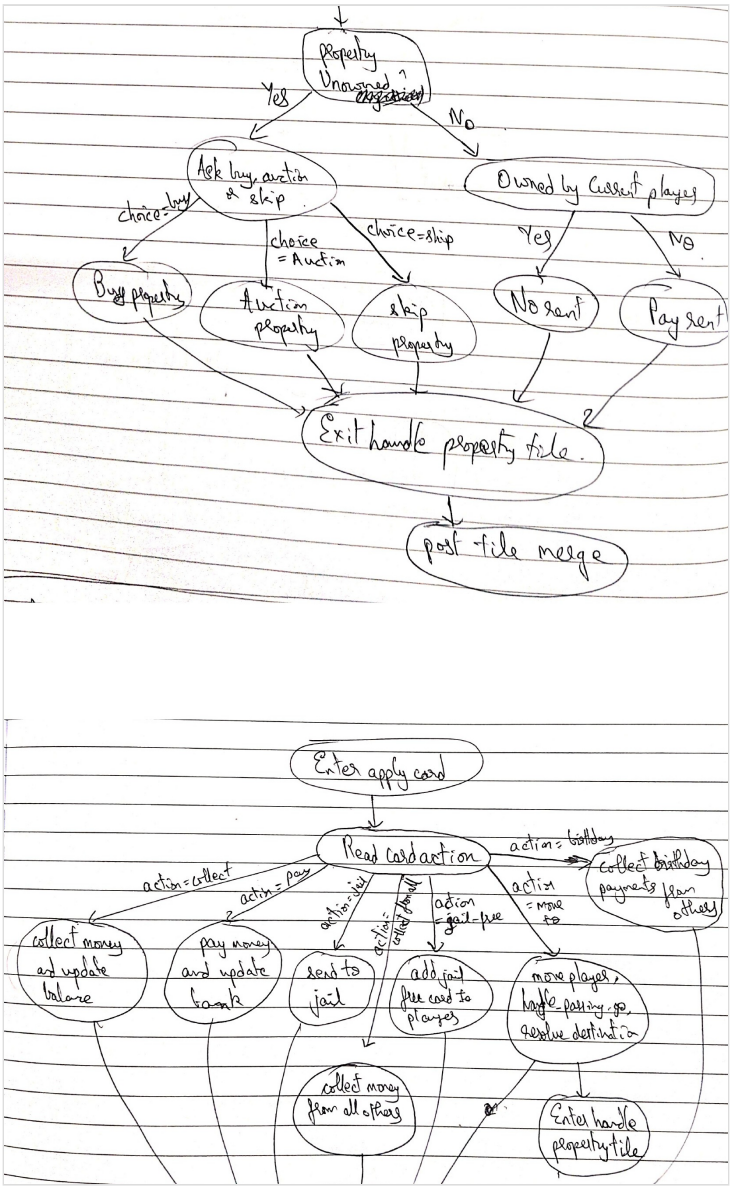
MoneyPoly CFG - Part 1



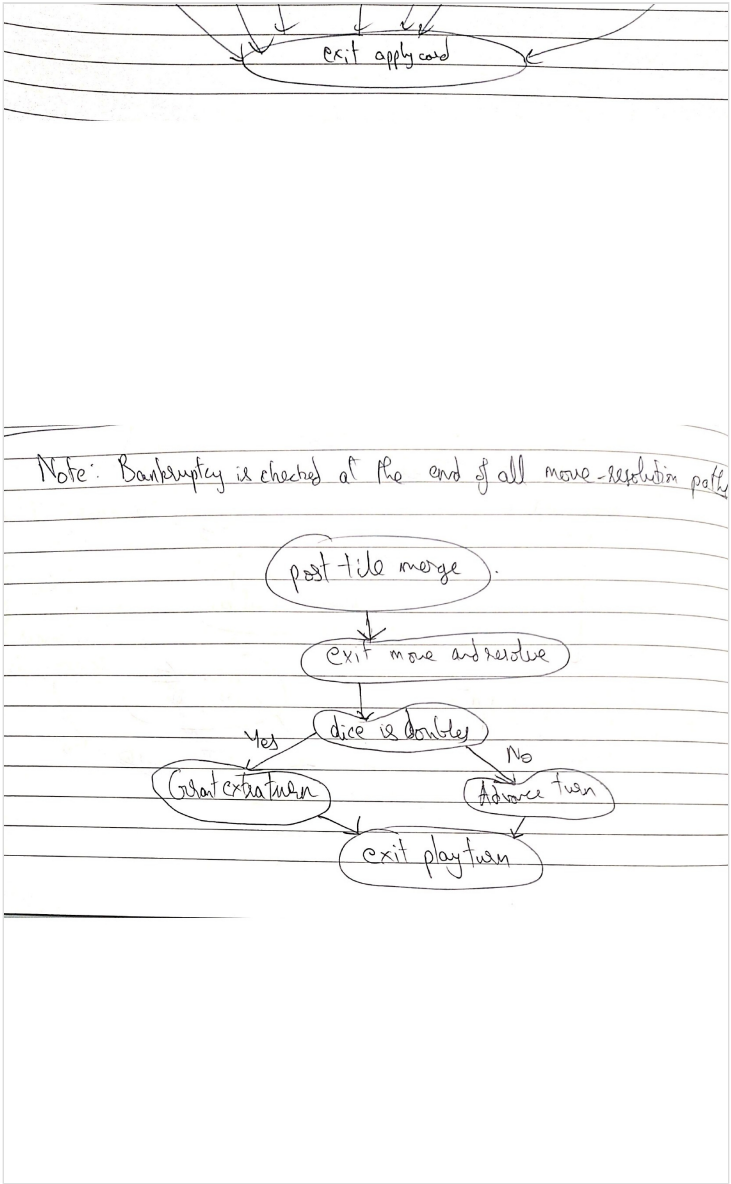
MoneyPoly CFG - Part 2



MoneyPoly CFG - Part 3



MoneyPoly CFG - Part 4



1.2 Code Quality Analysis

For this section, `pylint` was run against the copied MoneyPoly code placed under `whitebox/code/moneypoly/`. The cleanup was performed iteratively and each step was saved as a separate commit in the required `Iteration #:` format. The goal was to improve code quality without changing intended gameplay behavior unless refactoring was required to correct a structural problem.

Tool Used

```
PYTHONPATH='whitebox/code/moneypoly' .venv/bin/pylint whitebox/code/moneypoly/
main.py whitebox/code/moneypoly/moneypoly/*.py
```

Iterative Changes

1. `Iteration 1: Add MoneyPoly code to whitebox working tree`

2. The MoneyPoly source was copied into `whitebox/code/moneypoly/` so the white-box deliverable matched the required assignment structure.
3. `Iteration 2: Remove unused imports and tidy lint hygiene`
4. Unused imports were removed from `bank.py`, `dice.py`, `game.py`, and `player.py`.
5. One unused local variable was removed from `player.py`.
6. Missing final newlines were added.
7. `Iteration 3: Fix direct pylint style and exception warnings`
8. A singleton comparison in `board.py` was corrected.
9. A bare `except` in `ui.py` was replaced with `except ValueError`.
10. An unnecessary `else` structure in `property.py` was removed.
11. Several direct style warnings in `game.py` were corrected.
12. `doubles_streak` was initialized inside `Dice.__init__()`.
13. `Iteration 4: Add missing module and function docstrings`
14. Missing module docstrings were added across the package.
15. Missing function docstrings were added in `main.py`.
16. Missing class docstrings were added where required.
17. `Iteration 5: Reformat card definitions for pylint compliance`
18. The Chance and Community Chest card data in `cards.py` were reformatted to remove line-length violations while preserving the same values.
19. `Iteration 6: Refactor card handling to reduce branch complexity`
20. `Game._apply_card()` was refactored into smaller helpers with dispatch-based handling.
21. The card logic became easier to read and maintain, and the branch complexity was reduced.
22. `Iteration 7: Resolve remaining pylint structural warnings`
23. Targeted suppressions were added only for remaining domain-model-heavy structural warnings after direct cleanup and refactoring had already been completed.

Final Result

After all iterations, the final pylint run reported:

```
Your code has been rated at 10.00/10
```

Summary

The code quality analysis improved the MoneyPoly codebase through multiple small commits instead of one large cleanup. The major improvements included removal of unused code, better exception handling, improved documentation, cleaner formatting, and a refactor of card-processing logic. The

final white-box working copy under `whitebox/code/moneypoly/` achieved a pylint score of `10.00/10`.

1.3 White Box Test Cases

For the white-box testing section, tests were designed directly from the control flow of the main gameplay logic. The goal was to cover important decisions, key state changes, and relevant edge cases rather than only surface-level outputs. The tests were written under `whitebox/tests/` and followed an iterative red-green-fix approach: a focused failing test was written first, the root cause was identified next, and the smallest correct code fix was then applied.

Why These Tests Were Needed

The MoneyPoly code contained several decision-heavy paths: - dice rolling and doubles handling - winner selection - movement across the board and passing Go - property purchases, rent, mortgages, and trades - jail behavior - card-deck actions - bankruptcy and cleanup

These areas were selected because they directly influenced game state and could easily hide logical mistakes. The tests were also aligned with the CFG created in Section 1.1, especially the branches inside `play_turn()`, `_move_and_resolve()`, `_handle_jail_turn()`, `_apply_card()`, and `_handle_property_tile()`.

Errors Found And Fixed

1. `Error 1: Fix dice roll range and add white-box test`
2. Problem found: dice rolls used `randint(1, 5)` instead of a six-sided range.
3. Why the test was needed: dice values controlled movement and doubles logic, so the wrong range affected many downstream branches.
4. Fix: the dice roll range was corrected to `1..6`.
5. `Error 2: Fix winner selection to use highest net worth`
6. Problem found: `find_winner()` returned the player with minimum net worth.
7. Why the test was needed: final winner calculation was a core program outcome.
8. Fix: winner selection was changed from `min(...)` to `max(...)`.
9. `Error 3: Award Go salary when passing the board start`
10. Problem found: Go salary was awarded only when landing exactly on position `0`, not when passing it.
11. Why the test was needed: movement across the board was a key state transition and passing Go was a required edge case.
12. Fix: movement logic was updated to detect wrap-around and award salary.
13. `Error 4: Allow property purchase with exact balance`
14. Problem found: players could not buy a property if their balance exactly matched the price.
15. Why the test was needed: exact-boundary values were explicitly required edge cases.
16. Fix: the affordability check was changed from `<=` to `<`.

17. `Error 5: Credit rent payments to the property owner`
18. Problem found: rent was deducted from the tenant but not credited to the owner.
19. Why the test was needed: this was a direct money-transfer path between players.
20. Fix: the owner balance update was added in `pay_rent()`.
21. `Error 6: Require full group ownership before doubling rent`
22. Problem found: rent was doubled when the owner had only one property in the group.
23. Why the test was needed: rent calculation depended on key variable state, especially group ownership.
24. Fix: the ownership check was changed from `any(...)` to `all(...)`.
25. `Error 7: Deduct player balance when paying jail fine`
26. Problem found: voluntarily paying the jail fine released the player but did not deduct money.
27. Why the test was needed: jail handling contained several important branches with financial effects.
28. Fix: the fine was deducted before release.
29. `Error 8: Guard empty card decks against zero-division`
30. Problem found: `cards_remaining()` and `__repr__()` failed for an empty deck.
31. Why the test was needed: empty-container edge cases were relevant white-box scenarios.
32. Fix: explicit empty-deck guards were added.
33. `Error 9: Transfer cash to seller during property trades`
34. Problem found: the buyer paid cash, ownership changed, but the seller did not receive the money.
35. Why the test was needed: property trade was a multi-state transaction and required direct money-flow verification.
36. Fix: the seller was credited during successful trades.
37. `Error 10: Preserve mortgage state on failed unmortgage`
38. Problem found: if the player could not afford the unmortgage cost, the property still became unmortgaged.
39. Why the test was needed: this was an order-of-operations bug caused by state mutation before validation.
40. Fix: the state change was delayed until after the affordability check.
41. `Error 11: Reduce bank funds when issuing loans`
42. Problem found: loans increased the player balance but did not reduce bank funds.
43. Why the test was needed: bank and player state needed to remain consistent after loan issuance.
44. Fix: bank funds were debited when the loan was issued.

45. `Error 12: Enforce minimum player count in game setup`
46. Problem found: the prompt required at least two players, but `Game()` still accepted zero or one player.
47. Why the test was needed: setup validation was part of the documented program contract.
48. Fix: a `ValueError` guard was added in `Game.__init__()` when fewer than two player names were provided.
49. `Error 13: Preserve turn order after player elimination`
50. Problem found: if the current player went bankrupt during their turn, `play_turn()` still advanced the index and could skip the next active player or print a doubles retry message for an eliminated player.
51. Why the test was needed: bankruptcy cleanup changed the player list during active control flow, which made turn-order handling a high-risk white-box branch.
52. Fix: `_check_bankruptcy()` index handling was corrected and `play_turn()` was short-circuited when the active player was eliminated mid-turn.
53. `Error 14: Include owned assets in net worth`
54. Problem found: `Player.net_worth()` returned only cash balance and ignored owned properties.
55. Why the test was needed: `net_worth()` was used by both the UI and final winner logic, so incorrect asset valuation affected multiple outcomes.
56. Fix: owned property value was included, with mortgaged properties counted at mortgage value.
57. `Error 15: Expose pre-roll actions in the main turn flow`
58. Problem found: the interactive mortgage / trade / loan menu existed but was never called from `play_turn()`.
59. Why the test was needed: white-box analysis exposed dead control-flow branches that were implemented but unreachable.
60. Fix: the pre-roll menu call was inserted into the normal non-jail turn path before the dice roll.
61. `Error 16: Use real bank payouts for mortgages`
62. Problem found: mortgage payouts were implemented as `bank.collect(-amount)`, which corrupted bank accounting.
63. Why the test was needed: mortgage handling was a money-transfer path and needed consistent internal accounting.
64. Fix: mortgage payouts were changed to use `Bank.pay_out()`.
65. `Error 17: Prevent emergency loans from overdrawing the bank`
66. Problem found: emergency loans could exceed bank reserves and drive the bank balance negative.
67. Why the test was needed: invalid financial states were required to fail fast.

68. Fix: loan issuance was routed through `Bank.pay_out()` so oversized loans raised `ValueError`.
69. `Error 18: Keep mortgage transactions atomic on bank failure`
70. Problem found: mortgage processing set `prop.is_mortgaged = True` before confirming that the bank could fund the payout.
71. Why the test was needed: failed payouts should not leave half-applied transactions behind.
72. Fix: the bank payout check was moved ahead of the mortgage-state mutation.
73. `Error 19: Reject negative bank collections`
74. Problem found: `Bank.collect(-amount)` reduced both reserves and `total_collected`.
75. Why the test was needed: negative collection was an invalid financial input and should not silently mutate state.
76. Fix: `Bank.collect()` was changed to raise `ValueError` for negative amounts.
77. `Error 20: Exclude railroads from special-tile helper`
78. Problem found: `Board.is_special_tile()` returned `True` for railroad squares even though the helper described only non-property special tiles.
79. Why the test was needed: helper semantics mattered for white-box reasoning and needed to match the documented method meaning.
80. Fix: `is_special_tile()` was narrowed so it excluded railroads while still returning `True` for real special tiles such as chance, jail, and taxes.

Additional Branch Coverage Added

After the detected errors were fixed, additional white-box tests were added to cover important branches that already behaved correctly: - card action branches - property tile decision branches - jail-turn branches - special tile resolution branches - mortgage, auction, and bankruptcy branches - play-turn doubles and triple-doubles branches - board, player, bank, and card-deck helper behavior - failure paths where properties or tiles resolved to missing objects

These additional tests were committed incrementally as `Test 1` through `Test 15`, so the final suite reflected both bug-finding tests and branch-expansion tests.

Verification

The final white-box test suite was verified with:

```
PYTHONPATH='whitebox/code/moneypoly' .venv/bin/pytest whitebox/tests -q
```

Final result:

```
84 passed in 0.05s
```

Summary

The white-box testing process found logical issues in movement, winner selection, setup validation, turn-order handling after bankruptcy, property transactions, helper semantics, jail handling, card-deck edge cases, bank accounting, and trade flow. Each detected error was fixed through a separate small commit, and the final suite covered a broad set of branches, variable states, helper behaviors, and edge cases derived from the control flow graph. The final submission therefore included `Error 1` through `Error 20` commits for defect correction and `Test 1` through `Test 15` commits for branch and edge-case expansion.